🏢 Edulateral Foundation — Official Evaluative Assessment Bank

Day 2: Traditional ML, Parallel Computing, OpenMP, MPI & Intel oneDAL

Target Audience: GLA University 7th Semester B.Tech Computer Science & Engineering
Lead Trainer: Shivam Gupta | **Evaluation Mode**: Objective Technical MCQ Assessment
Total Questions: 25 Questions | **Time Limit**: 45 Minutes

Question 01: D2-Q01

In Flynn's Classical Taxonomy, how are modern shared-memory multi-core processors executing OpenMP multithreaded programs categorized?

- **(A)** SISD (Single Instruction, Single Data)
- **(B)** SIMD (Single Instruction, Multiple Data)
- **(C)** MISD (Multiple Instruction, Single Data)
- **(D)** MIMD (Multiple Instruction, Multiple Data)

> **Correct Answer**: **(D)**
> **Technical Explanation**: Modern multi-core processors running OpenMP are MIMD machines: each independent CPU core has its own program counter and executes distinct instruction streams on different data partitions, communicating through shared cache/memory.

Question 02: D2-Q02

What severe performance pathology occurs when two independent threads executing on separate CPU cores repeatedly write to distinct variables located on the exact same 64-byte L1 cache line?

- **(A)** Deadlock
- **(B)** False Sharing (Cache Line Bouncing)
- **(C)** DRAM Buffer Underflow
- **(D)** Paging Thrashing

> **Correct Answer**: **(B)**
> **Technical Explanation**: False sharing occurs when two threads modify distinct variables that reside within the same 64-byte hardware cache line. The MESI cache coherency protocol continuously invalidates the cache line across cores, forcing expensive main memory and L3 snoops despite the absence of an algorithmic race condition.

Question 03: D2-Q03

In OpenMP, which schedule type is optimal for parallel loops where the computational workload per iteration varies drastically (e.g., ray tracing or Mandelbrot fractals)?

- **(A)** schedule(static, 100)
- **(B)** schedule(dynamic, chunk_size)
- **(C)** schedule(runtime, static)
- **(D)** schedule(none)

> **Correct Answer**: **(B)**
> **Technical Explanation**: `schedule(dynamic, chunk_size)` assigns loop chunks to threads dynamically from a shared queue at runtime. As soon as a thread finishes its chunk, it requests another, preventing fast threads from sitting idle waiting for slow iterations to complete.

Question 04: D2-Q04

What is the key performance difference between `#pragma omp critical` and `#pragma omp atomic` in OpenMP?

- **(A)** `critical` is faster than `atomic` on all modern x86 chips.
- **(B)** `atomic` maps directly to hardware bus lock / atomic assembly instructions (e.g. `LOCK XADD`) for simple scalar updates, whereas `critical` uses a heavyweight software mutex lock.
- **(C)** `atomic` works across multiple Slurm cluster nodes; `critical` only works on laptops.
- **(D)** `critical` is for floating point numbers; `atomic` is only for strings.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `#pragma omp atomic` allows the compiler to generate single hardware atomic instructions (like x86 `lock cmpxchg` or `lock xadd`) with minimal cycle overhead. `#pragma omp critical` instantiates a global software mutex, serializing execution through OS locking primitives that carry significantly higher overhead.

Question 05: D2-Q05

What does the OpenMP environment variable `export OMP_PROC_BIND=close` enforce regarding thread affinity?

- **(A)** It shuts down all worker threads immediately.
- **(B)** It places consecutive OpenMP threads on adjacent physical cores contiguous to the master thread, maximizing L3 cache sharing.
- **(C)** It closes all open network ports.
- **(D)** It spreads threads across distant NUMA sockets to minimize heat.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `OMP_PROC_BIND=close` binds threads to contiguous processor cores as close as possible to the master thread. This is advantageous for workloads with heavy inter-thread data sharing, as threads can share data within common L2 or L3 caches.

Question 06: D2-Q06

In the Message Passing Interface (MPI), what is the function signature and behavior of `MPI_Bcast`?

- **(A)** It broadcasts an audio message to the computer speakers.

- **(B)** It broadcasts a contiguous buffer of data from a single designated root rank to every other rank within the specified communicator.
- **(C)** It sends a private encrypted email to the system administrator.
- **(D)** It splits the communicator into two equal sub-groups.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)` distributes data starting at `buffer` on rank `root` to all processes in communicator `comm`. It is a collective operation where all participating ranks must call the function.

Question 07: D2-Q07

What is the fundamental difference between `MPI_Scatter` and `MPI_Bcast`?

- **(A)** `MPI_Bcast` copies the exact same data block to all ranks; `MPI_Scatter` partitions a master array into disjoint equal chunks and sends one unique slice to each rank.
- **(B)** `MPI_Scatter` only works on GPUs; `MPI_Bcast` works on CPUs.
- **(C)** `MPI_Scatter` collects data into a single master rank; `MPI_Bcast` deletes it.
- **(D)** There is no difference; they are aliases for the same underlying function.

> **Correct Answer**: **(A)**
> **Technical Explanation**: In `MPI_Bcast`, every process receives a duplicate copy of the root's entire buffer. In `MPI_Scatter`, the root process divides an array of size $N \times P$ into P distinct segments of size N, sending segment i exclusively to rank i.

Question 08: D2-Q08

Why is non-blocking communication using `MPI_Isend` and `MPI_Irecv` accompanied by `MPI_Wait` considered superior to blocking `MPI_Send` / `MPI_Recv` in high-performance computing?

- **(A)** Non-blocking communication is immune to computer viruses.
- **(B)** It enables overlapping communication with computation, allowing the CPU to perform floating-point calculations while network interfaces transfer data asynchronously.
- **(C)** It removes the requirement for having InfiniBand network cards.
- **(D)** It converts C++ code into Fortran code automatically.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `MPI_Isend` and `MPI_Irecv` return control immediately before the network transfer completes. The CPU can then perform independent intensive computations while the InfiniBand HCA (Host Channel Adapter) handles packet transmission via zero-copy RDMA in the background, followed by `MPI_Wait` to synchronize.

Question 09: D2-Q09

What fatal condition occurs when Rank 0 executes a blocking `MPI_Recv` waiting for Rank 1, while Rank 1 simultaneously executes a blocking `MPI_Recv` waiting for Rank 0?

- **(A)** Cache Line Invalidation
- **(B)** MPI Deadlock
- **(C)** Hardware Interrupt Overflow
- **(D)** Kernel Panic

> **Correct Answer**: **(B)**
> **Technical Explanation**: This is a classic circular wait MPI Deadlock. Both processes block indefinitely waiting for an incoming message that neither process will ever send, freezing the cluster job until Slurm kills it upon walltime expiration.

Question 10: D2-Q10

What does the library `scikit-learn-intelex` (Intel Extension for Scikit-learn) do when activated with `from sklearnex import patch_sklearn; patch_sklearn()`?

- **(A)** It converts all Scikit-learn algorithms into neural networks.
- **(B)** It dynamically monkey-patches Scikit-learn algorithms with high-performance C++ kernels powered by Intel oneDAL, vectorizing mathematical operations with AVX-512 and multi-threading on Xeon CPUs.
- **(C)** It sends dataset samples to Intel cloud servers for training.
- **(D)** It restricts Scikit-learn to run on only one single thread.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `patch_sklearn()` replaces standard Python/Cython estimators (SVM, Random Forest, k-Means, PCA, k-NN, Ridge) at runtime with optimized oneDAL bindings. It requires zero code changes to the modeling logic while delivering 10x-50x speedups on Intel hardware.

Question 11: D2-Q11

What is Intel oneDAL (oneAPI Data Analytics Library)?

- **(A)** A cloud database for storing customer records.
- **(B)** A high-performance algorithmic C++ library designed for accelerated data analysis and machine learning across CPUs and GPUs, optimizing all stages from preprocessing to model training and inference.
- **(C)** A replacement for Apache Spark.
- **(D)** A Python package manager.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Intel oneDAL is the data analytics component of oneAPI. It implements highly optimized mathematical primitives (BLAS Level 3, SIMD vectorized kernels, tree-traversal algorithms) tailored to maximize CPU cache utilization and vector register throughput.

Question 12: D2-Q12

In Support Vector Machines (SVM), why does `sklearnex` achieve up to 30x-100x speedups over baseline `scikit-learn` (LIBSVM)?

- **(A)** It reduces the number of training epochs to 1.

- **(B)** It vectorizes kernel matrix evaluations (RBF/linear) using AVX-512 instructions, optimizes working set selection via multi-threaded SMO, and exploits cache blocking to fit sub-matrices in L2/L3 caches.
- **(C)** It converts the training data into low-resolution 8-bit integers.
- **(D)** It skips calculating the support vectors.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Standard LIBSVM relies on single-threaded, scalar evaluations of kernel distance matrices. oneDAL's SVM implementation leverages AVX-512 vectorization for kernel evaluations, multithreaded working set selection, and cache-blocked matrix operations.

Question 13: D2-Q13

How does `sklearnex` accelerate Random Forest training on multi-socket Intel Xeon servers?

- **(A)** By utilizing multi-threaded histogram-based split finding and vectorized feature binning across all physical NUMA cores.
- **(B)** By training only a single decision tree and duplicating it 100 times.
- **(C)** By converting decision trees into linear regression equations.
- **(D)** By deleting 50% of the input features.

> **Correct Answer**: **(A)**
> **Technical Explanation**: oneDAL builds decision trees using histogram-based feature binning vectorized via AVX-512/AVX2. Tree node splits and sub-tree construction are partitioned across threads with NUMA-aware memory allocations, eliminating inter-socket lock contention.

Question 14: D2-Q14

What happens if a user invokes `unpatch_sklearn()` in a Python script after patching?

- **(A)** Python crashes with a segmentation fault.
- **(B)** Scikit-learn estimators are restored back to their original stock implementations without restarting the Python interpreter.
- **(C)** The operating system uninstalls Scikit-learn completely.
- **(D)** The CPU frequency drops to 1 GHz.

> **Correct Answer**: **(B)**
> **Technical Explanation**: `sklearnex.unpatch_sklearn()` restores the original `sklearn` module attributes and estimator classes that were modified by `patch_sklearn()`, allowing side-by-side benchmarking within the same running script.

Question 15: D2-Q15

In OpenMP loop parallelization, what does the `reduction(+:sum)` clause prevent?

- **(A)** Memory leaks in dynamic memory allocation.
- **(B)** Data races and race conditions on the shared variable `sum` by giving each thread a private accumulator and combining them into the global variable at the end.

- **(C)** Compilation errors with GCC.
- **(D)** Excessive disk writes.

> **Correct Answer**: **(B)**
> **Technical Explanation**: Without `reduction(+:sum)`, multiple threads executing `sum += array[i]` concurrently would cause a data race (read-modify-write hazard). The `reduction` clause initializes a private copy of `sum` per thread and atomically sums them into the original variable upon exiting the parallel region.

Question 16: D2-Q16

What is the purpose of `MPI_Allreduce` in distributed scientific algorithms?

- **(A)** It performs an element-wise reduction (such as sum, min, max) across all ranks and distributes the final reduced scalar or array to every participating rank.
- **(B)** It reduces the clock speed of all nodes in the cluster.
- **(C)** It deletes all intermediate temporary files.
- **(D)** It shuts down idle compute nodes.

> **Correct Answer**: **(A)**
> **Technical Explanation**: `MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm)` is equivalent to calling `MPI_Reduce` to a root rank followed immediately by an `MPI_Bcast` to all ranks, but is implemented using optimal collective algorithms (like recursive doubling or Rabenseifner's algorithm).

Question 17: D2-Q17

In `mpi4py`, what is the difference between uppercase methods (e.g., `comm.Send()`, `comm.Recv()`) and lowercase methods (e.g., `comm.send()`, `comm.recv()`)?

- **(A)** Uppercase methods use fast direct memory buffers (C-contiguous NumPy arrays) via standard MPI buffer pointers; lowercase methods serialize arbitrary Python objects via pickle (incurring major CPU overhead).
- **(B)** Uppercase methods run on GPUs; lowercase methods run on CPUs.
- **(C)** Lowercase methods are deprecated and no longer supported.
- **(D)** Uppercase methods send messages in capital letters.

> **Correct Answer**: **(A)**
> **Technical Explanation**: In `mpi4py`, lowercase methods (`send`, `recv`, `bcast`) serialize general Python objects using `pickle`, which introduces massive serialization and memory copy latency. Uppercase methods (`Send`, `Recv`, `Bcast`) operate directly on C-contiguous memory buffers (NumPy arrays) with zero-copy C-level MPI speed.

Question 18: D2-Q18

What is the typical speedup observed when executing k-Means clustering on a 1-million sample dataset with Intel `sklearnex` compared to stock Scikit-learn?

- **(A)** 0.1x (slower)
- **(B)** 1.0x (identical)

- **(C)** 10x to 40x speedup depending on feature count and core allocation.
- **(D)** 10,000x speedup.

> **Correct Answer**: **(C)**
> **Technical Explanation**: Intel oneDAL's k-Means implementation vectorizes Euclidean distance calculations via AVX-512, maintains cluster centroids in L2/L3 caches, and parallelizes centroid updates across all threads, routinely yielding 10x to 40x speedups.

Question 19: D2-Q19

In OpenMP, what does `#pragma omp barrier` enforce?

- **(A)** It builds a software firewall to block unauthorized internet traffic.
- **(B)** It synchronizes all threads in a team; each thread waits at the barrier until all other threads in the team have arrived.
- **(C)** It terminates execution of the application.
- **(D)** It forces threads to write to the hard disk.

> **Correct Answer**: **(B)**
> **Technical Explanation**: A barrier is an explicit synchronization point. No thread can proceed past `#pragma omp barrier` until every thread in the current parallel team has reached the barrier, preventing race conditions across sequential computational phases.

Question 20: D2-Q20

What Intel oneAPI library provides optimized linear algebra routines (BLAS, LAPACK, FFT) that underpin Intel-optimized NumPy and SciPy?

- **(A)** Intel oneMKL (Math Kernel Library)
- **(B)** Intel Graphics Media SDK
- **(C)** Intel Quartus Prime
- **(D)** Intel Neural Compressor

> **Correct Answer**: **(A)**
> **Technical Explanation**: Intel oneMKL (Math Kernel Library) contains hyper-optimized BLAS, LAPACK, ScaLAPACK, and FFT routines hand-tuned for every generation of Intel processors, automatically dispatching AVX-512 and AMX instructions.

Question 21: D2-Q21

When parallelizing a 2D grid simulation in MPI, what technique is used to exchange boundary values between neighboring sub-domains?

- **(A)** Halo Exchange (Ghost Cell Exchange)
- **(B)** Kernel Paging
- **(C)** Memory Compaction
- **(D)** Round-Robin DNS

> **Correct Answer**: **(A)**
> **Technical Explanation**: In 2D/3D domain decomposition, each MPI rank processes a local sub-grid. To compute spatial stencils (like finite differences), ranks exchange border rows/columns with adjacent ranks in a process called 'Halo Exchange' (or Ghost Cells), typically implemented with non-blocking `MPI_Isend` / `MPI_Irecv`.

Question 22: D2-Q22

What is the effect of setting `KMP_BLOCKTIME=0` in an Intel OpenMP multithreaded application?

- **(A)** Threads yield the CPU immediately after completing their parallel region rather than busy-waiting, reducing CPU utilization during sequential phases.
- **(B)** Threads block for 0 milliseconds before terminating the application.
- **(C)** The compiler ignores all `#pragma` statements.
- **(D)** Memory bandwidth is throttled to zero.

> **Correct Answer**: **(A)**
> **Technical Explanation**: `KMP_BLOCKTIME` specifies the time (in milliseconds) a worker thread should wait in a spin-wait loop after completing a parallel region before going to sleep. Setting it to 0 causes threads to release the CPU immediately, freeing hardware cores for other processes.

Question 23: D2-Q23

Which MPI collective operation takes data from all processes and distributes the combined result to all processes (equivalent to an `MPI_Gather` followed by an `MPI_Bcast`)?

- **(A)** MPI_Allgather
- **(B)** MPI_Scatter
- **(C)** MPI_Barrier
- **(D)** MPI_Comm_split

> **Correct Answer**: **(A)**
> **Technical Explanation**: `MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)` collects data from all processes and stores the concatenated array in `recvbuf` on every participating process.

Question 24: D2-Q24

How does `scikit-learn-intelex` optimize Principal Component Analysis (PCA)?

- **(A)** By utilizing Intel oneMKL's vectorized SVD (Singular Value Decomposition) and correlation matrix eigensolvers with multi-threaded BLAS Level 3 matrix multiplication.
- **(B)** By randomly dropping 50% of the principal components.
- **(C)** By using a shallow decision tree.
- **(D)** By running on a single CPU thread with low clock speed.

> **Correct Answer**: **(A)**
> **Technical Explanation**: oneDAL accelerates PCA by computing the correlation/covariance matrix using oneMKL `GEMM` and finding eigenvalues/eigenvectors via LAPACK `syevd` / `gesvd` kernels optimized for AVX-512, avoiding Python loop bottlenecks.

Question 25: D2-Q25

What is the purpose of the MPI function `MPI_Init(int *argc, char ***argv)`?

- **(A)** It initializes the MPI execution environment and establishes network communication channels across participating cluster processes.
- **(B)** It formats the compute node hard drives.
- **(C)** It compiles the C++ code.
- **(D)** It powers on the server rack.

> **Correct Answer**: **(A)**
> **Technical Explanation**: `MPI_Init` must be called before any other MPI routines (except `MPI_Initialized`). It sets up internal communicator data structures, discovers network interfaces (InfiniBand/TCP), and establishes inter-process messaging handles.
